

AI 自適應學習平台 技術型高中版

系統現況與開發歷程

這個專案從我父親——一位技術型高中數學老師——每天遇到的教學問題開始。最早只是競賽中的 AI 數學原型；得獎之後我沒有停下來，而是一路改版、重構，把教材、數學題目、學生練習、AI、判題與學習紀錄一項一項做進同一套系統，最後把它帶回真正的課堂。

目前已進入真實課堂使用。115 學年第一學期起，系統於 2 個技術型高中班級正式上線，共 50 名學生；截至 2026-10-04，累積 1,224 次實際作答，平均答對率 67%。這些紀錄來自學生實際練習，不是測試資料。系統仍持續開發，教材匯入、GenCode、判題器與自適應研究也仍在逐步驗證。

2 個班級**50 名學生****1,224 次實際作答****67% 平均答對率**

多三甲 31 人 | 資一乙 19 人 | 真實學生使用紀錄 | 資料快照 2026-10-04

開發歷程

競賽原型

扶輪盃 AI 第一名，完成第一個能運作的版本。

重構擴充

育秀盃金獎後，持續補上教材、題目、介面與後端。

進入課堂

開始讓學生實際使用，問題也從這裡真正出現。

正式上線

115 學年第一學期起，2 班、50 名學生實際使用。

持續開發

教材匯入、GenCode、判題器與自適應研究同步進行。



網站教師首頁（正式上線） 兩個真實班級、50 名學生，以及教材、技能與 AI 的管理入口。

公開唯讀 Demo（不會寫入正式資料）

mathb.math-adaptive.org/review-demo

我一開始想解決什麼問題？

我的父親是技術型高中數學老師，有二十多年教學經驗。我不明白老師需要什麼，所以直接問他：「如果真的拿到班上，你會不會用？」「哪裡不好用？」「真正需要的是什麼？」後來我發現，推動這套系統長大的不是一張完整藍圖，而是一路遇到的真實問題。

1 一整冊教材，難道每一道題都要人工輸入？

後來做成教材匯入系統，整理成冊、章、節、技能與例題資料。

2 如果題目都是固定的，學生怎麼持續練習？

後來把「一題」變成「一支會出題的程式」。

3 AI 可以產生題目，但我怎麼知道它沒有錯？

後來建立生成、檢查、修正、測試到發布的流程。

4 學生做數學需要公式、手寫、作圖與提示，普通表單不夠。

後來做成數學練習介面，並加入 AI 助教。

分工 | 葉陽甫

系統架構與程式開發、前後端、教材匯入流程、出題程式（GenCode）、數學判題、AI 整合、資料庫、測試與除錯、正式部署與維護。

父親／技高數學教師

提供教學需求與課堂情境、課本資料匯入與整理，試用系統並回饋哪裡不好用。

第三階段研究・科展與旺宏科學獎

系統已經會出題了，但它知道「為什麼下一題要出這一題」嗎？

這個問題後來成為我自適應學習研究的核心：學生作答 → 找出弱點 → 決定下一步 → 補救練習 → 回到學習主線。研究已啟動，尚未整合進正式系統。前面是工程問題，這裡開始是研究問題，但題目一直沒有換。

一本課本，不是把 PDF 上傳就完成了

數學教材不是單純文字：有冊、章、節、技能，有例題、隨堂練習、習題與自我評量，還有公式、特殊符號與圖形。如果匯入後這些關係消失，之後就沒辦法可靠地產題、判題，也找不到學生在哪個技能出錯。只把 PDF 丟給 AI，就會這樣。

所以我做的不是檔案上傳，而是教材匯入與整理流程：同時讀 Word 與 PDF，把教材拆成冊、章、節、技能與每一道題，並記住每題的來源。



AI 教材匯入系統 V3

Word 精準結構 × PDF 完整視覺

匯入教材內容

教材來源

Word + PDF，用於匯入教材內容、題目、公式與圖片。

已選擇此模式

匯入教材目錄

PDF only

僅用 PDF，用於建立章節結構與正式章節名稱。

前往匯入目錄

1 教材設定

課程 (Curriculum)

技高 (Vocational)

年級 (Grade)

10

出版社 (Publisher)

龍騰 (Longteng)

冊別 (Volume)

數學B2

2 上傳教材

Word 教材

結構與數學內容來源

匯入教材文字、表格、MathType / Equation Editor 公式。

選擇檔案 未選擇任何檔案

PDF 教材

圖片與視覺內容來源

保留圖例圖、照片、座標圖及完整教材視覺版面。

選擇檔案 未選擇任何檔案



例題管理中心

新增例題

課程篩選器

課程: 技高 年級: 10 冊別: 數學B1 卷: 第1卷 系統系與函數圖形 1-1 數線與絕對值 清除

系統錯誤訊息: 本頁錯誤-10, formula assets 錯誤-0, FORMULA IMAGE 錯誤-0, FORMULA MISSING 錯誤-0, MATH_PARSE_FAILED 錯誤-0, MATH_PARSE_FAILED tokens-0, image_placeholder_no_asset 錯誤-0, missing_no_asset 錯誤-0

ID	技能	內容	來源	Gencode V3	操作
4398	絕對值	數線上，若 $ x = 6$ ，試求 x 之值。	1-1習題 基礎題 1	已上傳	編輯 刪除
4399	絕對值	已知數線上兩點 $A(-3)$ 、 $B(7)$ ，試求 A 、 B 兩點的距離。	1-1習題 基礎題 2	已上傳	編輯 刪除
4408	絕對值	數線上，若 $ x = 7$ ，試求 x 之值。	例1	已上傳	編輯 刪除

例題管理中心 匯入後拆成章、節、技能與每一道題，每題保留原始來源（如「1-1 習題 基礎題 1」）與出題程式狀態。

匯入系統 V3 選課綱、年級、冊別，上傳 Word 與 PDF。Word 保留結構與公式，PDF 保留完整視覺。

匯入時要保住的四件事

01 | 保留章節結構

02 | 辨識數學公式

03 | 保留教材圖片

04 | 記住每題來源

技術型高中數學 B1 · B2 · B3 · B4

四冊教材進入同一套資料架構，每一道題都知道自己來自哪一冊、哪一章、哪一節與哪個技能。並不是每一題都已完成練習化，每一章仍分別驗收。

我把「一題」變成「一支會出題的程式」

傳統題庫是「題目＋答案」，學生做完就只是一題。要大量練習，不可能每一道變化題都人工建立。我後來發現，能一直擴充的題庫不應該只保存「題目」，還要保存「這一類題目怎麼產生」，我把這個做法叫「程式即內容」。

流程

1 →	2 →	3 →	4 →	5
從教材例題分析題型	建立出題程式 (GenCode／Generator)	每次改變數值或條件，產生同類新題	程式自己算出正確答案	判題器檢查學生的答案

固定題庫：一題、一個答案，做完就結束。

出題程式：同一類題目可以產生很多數值不同的題，答案不靠人工對。

教材例題 ID
4404

技能名稱
絕對值不等式

SKILL ID
vh_數學B1_AbsoluteValueInequality

元件 ID (COMPONENT)
src_4404

目前載入來源
PRODUCTION

是否已同步學生端
是

最後生成時間
2026-08-12 12:49:18

絕對值不等式

4404

vh_數學B1_AbsoluteValueInequality

解不等式 $|4x + 1| \leq 6$ 。

題目內容

解不等式： $|7x - 2| < 6$ 。

✓ 正確答案

$(-\frac{4}{7}, \frac{8}{7})$

題目內容

解不等式： $|2x + 14| \leq 5$ 。

✓ 正確答案

$[-\frac{19}{2}, -\frac{9}{2}]$

題目內容

解不等式： $|3x + 3| \leq 3$ 。

✓ 正確答案

$[-2, 0]$

同一支程式，不同的題目 教材例題 4404「絕對值不等式」的出題程式（已上線），產生三道數值不同的題。

在系統裡，這一層負責出題、精確運算、建立標準答案與判斷對錯，由 Domain、Generator 與判題器組成。它不依賴 AI 的回答，因此可以用測試去驗證，這也是下一頁需要驗證流程的原因。

生成式 AI 產題的可靠性問題與驗證流程

一開始很容易想成「教材 → AI → 題目 → 學生」。真的做下去才發現：公式會壞、答案對不起來、只有某些隨機數字才出錯、程式可能根本跑不起來、圖形可能和答案不一致，改一個地方還可能弄壞原本正常的題型。所以 AI 只負責寫出題程式的草稿，後面每一步都是為了一個具體的錯誤而設。

流程與每一步的理由

1 AI 協助產生程式	AI 負責草稿，不負責保證正確。
2 實際執行	因為程式可能根本跑不起來。
3 隨機產題試跑	只有某些數字會出錯，要跑很多次。
4 檢查題目與答案	公式、答案、圖形必須一致。
5 自動修正常見錯誤	重複出現的格式與語法問題（研究中實作的 AST Healer）。
6 pytest 回歸測試	改一處，不能弄壞其他題型。
7 人工驗收	最後由人看過再發布。
8 發布到學生端	通過上面所有檢查才上線。

另外程式在隔離環境執行，避免出題程式影響主系統。

目前狀態：已上線	
最後生成時間：2026-08-12 12:49:10	
▼ 進階資訊	
skill_id	vh_數學B1_AbsoluteValueInequality
total_examples	10
verified_count	10
failed_count	0
unsupported_count	0
published_count	10
not_packaged_count	0
missing_tracker_count	0
production_wrapper_exists	yes
v3_package_exists	yes
generator_specs_count	10
production_component_count	10

驗證與發布結果（正式系統） 10 個教材元件，10 個驗證通過、0 個失敗、10 個正式發布。

這是「絕對值不等式」單一技能於 2026-08-12 的驗收結果，不代表全部教材都已完成。

AI 能產生，不等於題目可以上線。只有通過執行、答案檢查、修復、測試與人工驗收的題目，才會進入學生使用的正式系統。

數學計算與語言教學的分層設計

學生做數學需要公式、手寫、作圖與提示。我沒有讓 AI 一個人教數學，而是把系統分成兩層：數學的正確性由程式負責，自然語言的解釋與引導由 AI 助教負責。

第一層 | 數學計算

Domain · Generator · 判題器

我們先把教材題型轉成可執行的 Python 出題程式。每次程式改變數值或條件產生新題時，也會同步依相同參數計算出該題的正確答案，再交由判題器比對學生作答。題目與答案來自同一套可執行規則，而不是由 AI 分別生成，因此數學邏輯可以透過測試反覆驗證。

Python 出題程式 → 產生變化題 + 同步計算答案
→ 判題器

第二層 | 語言教學

AI 助教

解釋題意、分段提示、分析學生錯在哪，用觀察、聯想、執行三步提問引導學生自己想。

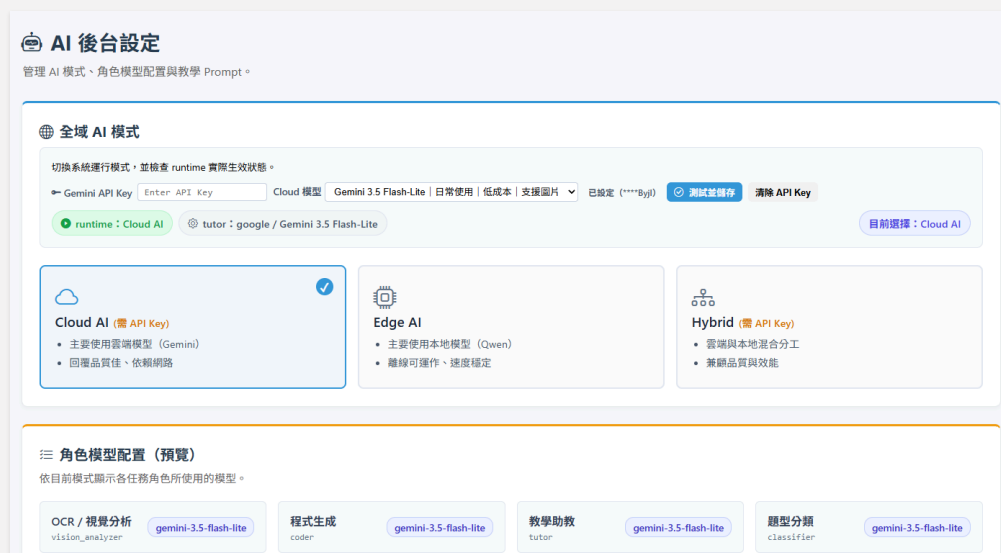
The screenshot displays a student practice interface. On the left, a problem is presented: "解不等式: $|5x - 3| < 13$." Below it is a text input field for the answer and a "提交" (Submit) button. A toolbar with icons for drawing and editing is visible. The main area shows a hand-drawn solution: $-13 < 5x - 3 < 13$, $-16 < 5x < 16$, and $-\frac{16}{5} < x < \frac{16}{5}$. On the right, the AI assistant provides guidance. It starts with a greeting "你好！有問題問我～" and a prompt "【執行】第一步應該怎麼做？請只提示，不要直接解答。". It then explains the key is to use the definition of absolute value to decompose the inequality. It asks the student to think about the range of the expression when the absolute value is less than a positive number. It provides a tip to remove the absolute value symbol and write the inequality in interval form. Finally, it points out that the student's reasoning or simplification still needs adjustment, showing their current work: $-13 < 5x - 3 < 13$; $-16 < 5x < 16$; $-16/5 < x < 16/5$. It gives a feedback message: "錯誤提示：式子格式或記號不完整，請先補齊括號、變數與表示方式。" and identifies the problem type as a general inequality. It concludes with a specific instruction: "具體下一步：請先補齊括號、變數與等號兩側的式子結構，再依題意繼續化簡。"

學生練習介面（正式網站） 題目、答案欄與手寫畫布由程式出題與判題；右側 AI 助教讀了學生的手寫算式，只給提示，回覆前程式會先檢查有沒有洩漏答案。

正式練習時，題目、標準答案與判分主要由第一層負責，AI 不直接當作數學答案的來源。把數學邏輯與語言生成分開，是為了降低生成式 AI 影響數學正確性的風險；這只是降低風險，不代表 AI 助教不會出錯，因此提示內容仍需要持續檢查。

模型調度與部署考量

學校和 Demo 不一樣：少數人試用時，每次都呼叫大型雲端模型看不出問題；數十位學生長期使用後，API 費用、回應速度與資料外傳都會變成實際問題。所以我讓系統可以依任務選擇模型，而不是綁死一種。



AI 後台設定（正式系統） 全域切換 Cloud／Edge／Hybrid，並依角色（視覺分析、程式生成、教學助教、題型分類）分別配置模型。

能力

教材理解、程式生成與多模態分析需要較強的雲端模型；簡單提示與固定範圍的互動，小型本地模型有機會取代部分雲端呼叫。

隱私

可在本機完成的互動留在本機，減少作答與學習資料不必要的外傳。這是減少，不是絕對保證。

已實作

雲端模型串接（Gemini）、Cloud／Edge／Hybrid 全域切換、依角色配置模型、本地模型（Qwen）選項、本地主機部署。

實作後我發現，模型選擇不是單純比較哪一個模型比較強，而是不同任務有不同需求。程式生成可以接受較高延遲，但學生在等待提示時，回應速度就很重要；涉及學生作答與學習紀錄時，也要考慮這些資料是否真的有必要送到雲端。因此 Cloud、Edge 與 Hybrid 的設計，是在能力、延遲、成本與資料傳輸之間做取捨。

成本與延遲

雲端模型有 API 成本與網路延遲；本地模型沒有 API 費用，但取決於主機效能與推論速度。

三種模式

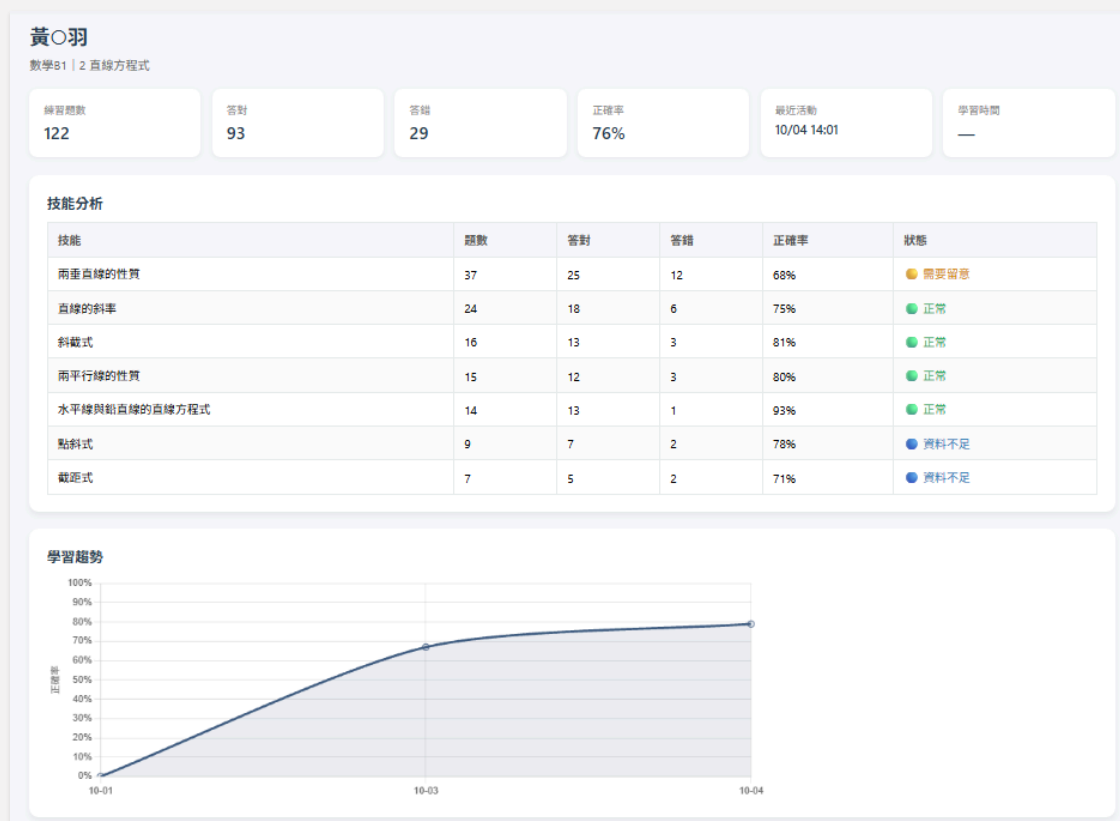
Cloud 全用雲端；Edge 全用本機；Hybrid 簡單任務先在本機完成，需要時再交給雲端。

仍在驗證

不同本地模型的品質與速度、混合調度與 fallback 策略、成本與品質的取捨。目前尚未證明小型模型足以取代雲端。

每一次作答，都留下學習紀錄

我不想只記錄分數，而是希望看見學生在哪裡卡住。每次作答都會被記錄，累積成各技能的題數、正確率與時間趨勢，讓學生與老師知道哪個觀念已經熟悉、哪裡還需要再練。



學生個人學習分析（正式網站，2026-10-04） 數學 B1「直線方程式」：122 題、正確率 76%。姓名已遮蔽。

技能拆開看

不只看總分，把每個數學技能分開分析，標出需要留意、正常與資料不足。

看見變化

比較不同時間的正確率，看學生是否隨練習進步。

目前的限制

這些紀錄目前用於呈現，還沒有用來自動決定下一題。

同樣的紀錄也能匯整成班級視角，教師可以看到練習題數、答對率、最近活動與需要留意的學生。這些資料是後面自適應研究的材料。

目前這些紀錄主要用於學生與教師查看，還沒有直接決定下一題。也就是說，系統現在已經能「看見學習歷程」，但還沒有完全做到「根據歷程做決策」。後面的自適應研究，就是希望把這些作答資料轉成模型可以使用的學生狀態，再進一步決定補救內容與下一步練習。

真的有人用之後，問題才真正開始

有些問題不是測試資料能發現的，學生真的開始作答後才會暴露。下面是三個實際遇過的例子，每一個都是先出錯，才被修進系統。

1 生成的題目本身有錯

課堂上學生遇到一題要求「求 $a+b$ 」，題目裡卻沒有定義 b ，是生成內容少了必要參數，不是畫面問題。

解決 回頭檢查出題程式與題目規格，確認題幹要求的變數都有定義。

之後 補上回歸測試，同類出題程式每次修改都重新驗證參數完整。

2 答案正確卻被判錯

學生會用分數、小數、區間、LaTeX 或不同順序寫同一個答案，只比文字會誤判。

解決 先把答案正規化，再判斷數學上是否相等。

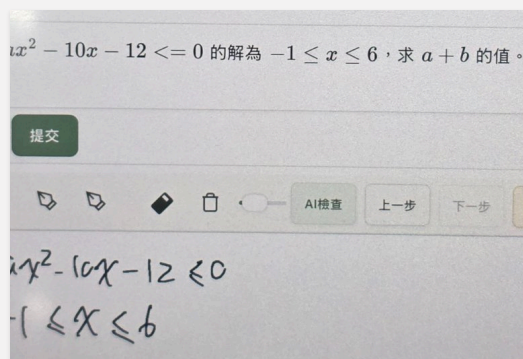
之後 曾判錯的答案都加入回歸測試。

3 作圖題不是填充題

答案可能畫在數線、坐標平面或手寫區，不是文字框。

解決 把作圖題視為獨立作答類型，使用專用規格與判題器。

之後 不再硬轉成文字答案判斷。



現場錯題：要求 $a+b$ ，卻只有參數 a 。



實際進班使用。



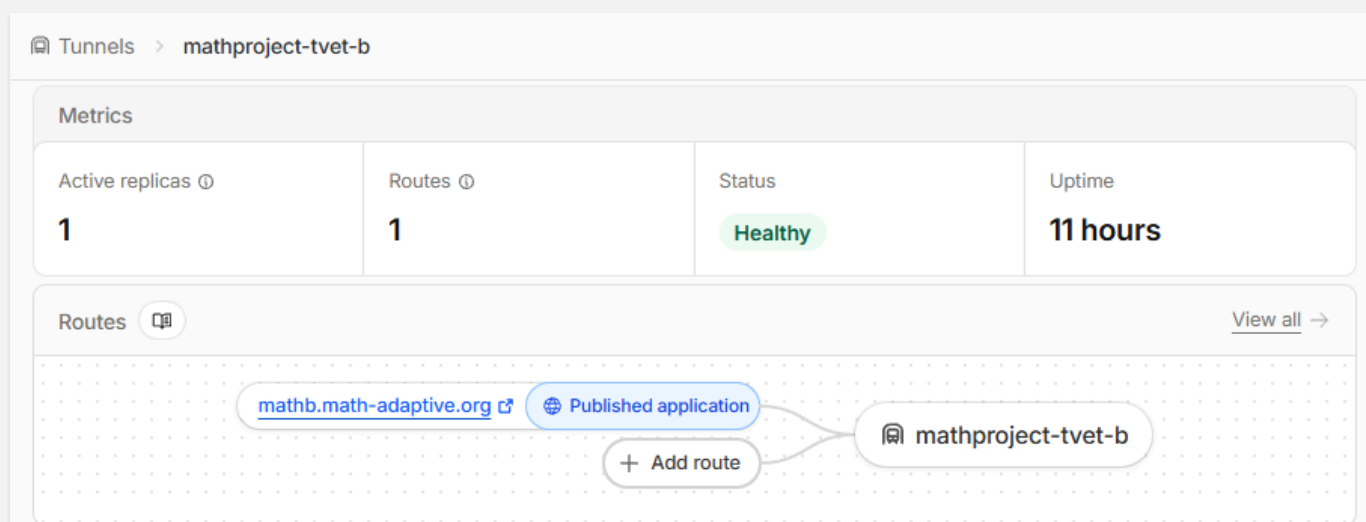
學生一起操作與討論。

2 個班級、50 名學生、1,224 次作答，都是學生使用留下的真實紀錄，不是展示資料。

真正困難的不是把功能做出來，而是讓它在不同教材、題型與學生的真實操作下仍然可靠。每一次現場出錯，最後都變成一條不能再犯的回歸測試。

從 localhost 到外部正式服務

一開始系統只跑在我自己的電腦上。要讓學生從學校或家裡連進來，我才開始處理主機、網路、網域與外部連線。現在正式主機放在家中書房，用家裡的電腦當伺服器，透過 HiNet 網路與 Cloudflare Tunnel 對外提供服務。localhost 跑得動，跟學生真的連得進來，是兩件不同的事。



Cloudflare Tunnel（真實後台） Tunnel「mathproject-tvet-b」狀態 Healthy，正式網域 mathb.math-adaptive.org 已接上。主機 IP、Replica ID 與主機名稱已裁除。

連線路徑

學生／老師 → 網際網路 → **Cloudflare（DNS＋Tunnel）** → **HiNet 500M** → 家中書房主機 → **Python／Flask** → 教材・題庫・資料庫・AI

部署時要處理的事

主機長時間運作、網域與 DNS、Tunnel 對外連線、Flask 服務與資料庫的啟動與維護。

目前的限制

正式主機只有這一台，在家中書房。同時使用人數也只驗證到目前 2 個班級的規模。

主機規格（僅供參考）：AMD Ryzen 7 5700F、32 GB RAM、RTX 5060 Ti 16 GB、HiNet 500M。

GitHub | 系統程式碼與版本紀錄

github.com/yyfsunnyboy/Mathproject_tvet_mathB

目前完成、正在研究、尚未完成

目前完成 | 已上線使用

以技術型高中數學 B1-B4 為基礎：教材匯入、程式化出題、AI 產題與驗證、數學判題、手寫／圖片／作圖練習介面、AI 助教、教師端、學習紀錄、正式網站。已有 2 個班級、50 名學生、1,224 次作答。

正在研究 | 研究已啟動，尚未整合進正式系統

自適應學習研究是我目前科展與旺宏科學獎的主題，已獲東區科展優等、旺宏科學獎決賽入選。四個角色：

AKT 診斷學生目前學到哪裡

PPO 決定下一步學什麼、練什麼

RAG 找到要補強的教材與觀念

GenCode 提供可驗證的練習題

目標是形成診斷 → 決策 → 補救 → 練習 → 再診斷的閉環。研究與工程同步進行，下一步是把成果接回正式系統。

尚未完成

自動退回前置技能、完整的個人化學習路徑、補救後自動回到原本主線、完整自適應閉環的正式上線，以及對學習成效的驗證，都還沒有完成。教材範圍也還只有技高數學 B；國中、普高與技高數學 A・C 是之後要用同一套架構擴充的方向，尚未開始。

現在的正式系統已經能產生題目、判斷答案、記錄作答，也累積了真實學生資料；但「有資料」不等於「知道下一步要怎麼教」。AKT、PPO 與 RAG 目前處理的，就是從這些作答歷程推估學生狀態、決定下一步，再找到需要補強的教材。這些模型已有研究與實驗，但尚未形成正式系統中的完整閉環。

前兩個階段解決的是工程問題：教材怎麼進來、題目怎麼生成、答案怎麼判斷、師生怎麼使用。研究要回答的是：系統能不能從作答歷程判斷學生狀態、安排下一步，並在補救後把學生帶回主線。目前這還沒有被證明。

下一階段的研究問題

高中階段，我從模型、系統實作一路做到真實學生使用。下一階段，我想建立可重現的實驗、擴大使用場域，並驗證系統是否真的能改善學習。

「這套系統真的能讓學生學得更好嗎？」

要回答它，還需要補上的六件事

1 可重現的實驗

不同模型、策略與版本可以重複比較，並保留完整實驗紀錄。

2 擴大系統測試

從約 50 人，測試多人同時使用時的穩定性、效能與資料一致性。

3 學習實驗設計

未來需要建立前測、後測與可比較的分組條件，避免只看到學生練習後分數提高，就直接認定是系統造成。

4 資料分析

除了總答對率，我希望比較補救前後的進步幅度、完成技能所需時間、同類錯誤是否重複，以及不同學生是否得到一致效果。

5 研究倫理與資料治理

若擴大到更多班級，學生學習歷程、作答內容與模型分析結果都需要更完整的匿名化、同意與資料使用規範。

6 擴大使用場域

進入更多班級與學習情境，檢驗結果能否重現。

研究模型 → 系統落地 → 擴大場域 → 實驗驗證 → 教育成效

目前尚未證明學習成效，這是下一階段要做的事。高中階段我完成了系統實作與初步的真實使用；接下來需要更完整的實驗方法與長期驗證。